

Stato dell'arte degli strumenti e delle tecniche di rilevazione del plagio del codice sorgente

Valerio Pio De Nicola

*Dipartimento di Scienze, Università di Bologna
valeriopio.denicola@studio.unibo.it*

Abstract

Qui ci andrà l'abstract

1 Introduzione

1.1 Contesto e Motivazione

Negli ultimi decenni, lo sviluppo del software è diventato una componente ubiqua e critica in quasi ogni settore industriale e accademico. Parallelamente alla crescita della domanda di software, si è assistito a un aumento esponenziale dell'educazione informatica e delle risorse disponibili online, tra cui repository open source come GitHub e forum come Stack Overflow. Più recentemente, la diffusione di strumenti di AI generativa (come GitHub Copilot e i modelli della famiglia GPT) ha ulteriormente abbassato la soglia di accesso alla produzione di codice, introducendo nuove forme di riutilizzo non autorizzato che sfuggono ai paradigmi classici del plagio. Questo scenario ha reso il codice sorgente una risorsa estremamente accessibile, facilitando tuttavia pratiche scorrette come il plagio del codice sorgente (*Source Code Plagiarism*).

Le conseguenze di questa pratica si manifestano su due fronti distinti. In ambito accademico, il plagio mina l'integrità del processo valutativo e ostacola l'apprendimento degli studenti, che non acquisiscono le competenze logiche necessarie [1]. In ambito industriale, il riutilizzo non autorizzato di codice protetto da copyright solleva gravi questioni legali legate alla proprietà intellettuale e alla violazione delle licenze software. La rilevazione manuale di questi casi è diventata un compito impraticabile a causa della vastità dei dati da analizzare, rendendo lo sviluppo di strumenti automatizzati per la **Source Code Plagiarism Detection** (SCPD) un campo di ricerca attivo e necessario.

1.2 Definizione del problema

Il plagio del codice sorgente consiste nel riutilizzo non autorizzato di frammenti di codice altrui, presentandoli come propri senza adeguata attribuzione [2]. A differenza del plagio testuale, il codice possiede una struttura sintattica rigida e una semantica operativa: gli strumenti di rilevazione devono essere consapevoli del linguaggio

di programmazione per distinguere tra *boilerplate code* (codice standard necessario) e logica originale. Questo rende il problema intrinsecamente più complesso rispetto al confronto tra testi in linguaggio naturale.

Un'ulteriore difficoltà risiede nel fenomeno dell'offuscamento: chi plagia può applicare trasformazioni sistematiche al codice originale (dalla semplice ridenominazione delle variabili fino alla riscrittura completa dell'algoritmo) per mascherare l'origine del materiale e sfuggire ai controlli automatici. La tassonomia di queste trasformazioni e le tecniche per contrastarle costituiscono il nucleo del presente studio, approfondito nelle sezioni 2 e 3.

1.3 Obiettivi dello studio

Questo studio mira a fornire una rassegna sistematica e aggiornata sullo stato dell'arte degli strumenti e delle tecniche per la rilevazione del plagio del codice sorgente (SCPD), identificando i metodi principali e le loro limitazioni. Gli obiettivi specifici includono:

- **Rassegna delle Tecniche:** Catalogare approcci teorici (string-based, tree-based, graph-based, learning-based) e valutarne efficacia contro varianti offuscate e codice AI-generato
- **Analisi Strumenti:** Esaminare tool open-source (es. JPlag, Dolos) e commerciali, producendo tabelle comparative su supporto linguaggi, accuratezza e usabilità
- **Valutazione Metodologica:** Definire metriche (precision, recall, robustness) e dataset standard, identificando sfide emergenti come il plagio cross-language e da IA generativa

2 Il Plagio nel Codice Sorgente: Tassonomia e Offuscamento

È importante analizzare la natura del plagio del codice sorgente, distinguendolo dal plagio testuale e classificando le diverse strategie utilizzate per occultare il riutilizzo non autorizzato di codice.

2.1 Cos'è il Plagio Software

Il plagio del codice sorgente è definito come l'atto di riutilizzare frammenti di codice altrui senza autorizzazione o attribuzione, presentandoli come propri. Nel contesto accademico è utilizzato come mezzo per ottenere vantaggi valutativi, ostacolando l'acquisizione di competenze logiche. Nel contesto professionale invece, si presenta spesso come una violazione delle licenze software e diritti di proprietà intellettuale.

A differenza dei contenuti testuali, il codice ha una struttura sintattica rigida e una semantica operativa. Gli strumenti di rilevazione devono essere «consapevoli» del linguaggio di programmazione per distinguere tra boilerplate code (codice standard necessario) e logica originale [3]. L'integrazione di risorse come Stack Overflow e repository open source ha reso i confini dell'utilizzo lecito sempre più sfumati, complicando ulteriormente la rilevazione del plagio [4].

2.2 Tassonomia del plagio nel codice sorgente

Per analizzare il fenomeno, è utile classificare il plagio in base al livello di modifica apportato al codice originale. Una tassonomia efficace, supportata da **Hage et al. (2010)** [5] e **Prado et al. (2019)** [6] prevede quattro livelli crescenti di complessità:

- **Plagio Letterale (Verbatim):** Copia esatta del file. Le uniche modifiche riguardano elementi non funzionali come spazi bianchi o commenti.
- **Plagio Superficiale (Shallow):** Include ridenominazione sistematica di variabili e il cambiamento dell'ordine di dichiarazioni che non influenzano il flusso del programma.
- **Plagio Strutturale:** Comporta una modifica della logica di controllo, come la trasformazione di un ciclo `for` in un ciclo `while`, o l'estrazione di blocchi di codice in funzioni separate.
- **Plagio Semantico/Logico:** La riscrittura completa dell'algoritmo. Il comportamento osservabile rimane identico, ma la forma cambia radicalmente (es. passare da una funzione iterativa a una ricorsiva).

Nella letteratura sui code clones [13], questa tassonomia viene frequentemente espressa attraverso una notazione numerica equivalente: il plagio letterale corrisponde al **Tipo 1**, quello superficiale al **Tipo 2**, quello strutturale al **Tipo 3** e quello semantico al **Tipo 4**. Nel seguito del presente lavoro le due nomenclature verranno utilizzate in modo intercambiabile, privilegiando la notazione numerica nelle analisi comparative.

2.3 Livelli e tecniche di offuscamento

L'offuscamento è l'applicazione intenzionale di trasformazioni per rendere il codice difficile da comprendere per

gli esseri umani e difficile da analizzare per gli strumenti automatici.

2.3.1 Offuscamento lessicale e sintattico

L'offuscamento lessicale agisce sulla superficie testuale del programma, cercando di nascondere l'origine del codice attraverso modifiche formali che non toccano la logica interna. Una delle strategie più diffuse è lo **scrambling**, ovvero la ridenominazione degli identificatori, in cui nomi di variabili, funzioni e classi vengono sostituiti con nomi casuali o privi di senso, rendendo la lettura del codice difficile per un operatore umano.

A questa pratica si affianca spesso l'inserimento di «codice morto»: si tratta di istruzioni sintatticamente corrette ma funzionalmente irrilevanti, inserite al solo scopo di «sporcicare» l'impronta digitale del file e ingannare gli algoritmi di confronto. Sebbene questi attacchi mettano seriamente in crisi approcci di confronto testuale puro (string-based), essi risultano spesso inefficaci contro i sistemi basati su token, i quali, durante la fase di normalizzazione, eliminano commenti, spazi e nomi degli identificatori per concentrarsi sulla struttura dei tipi di istruzione.

2.3.2 Offuscamento strutturale

Un livello di manipolazione più avanzato mira a nascondere la gerarchia ed il flusso di esecuzione del programma attraverso trasformazioni strutturali. Tecniche comuni includono l'**inlining** e l'**outlining** delle funzioni: nel primo caso, il corpo di una funzione viene espanso direttamente nel punto di chiamata; nel secondo, un blocco di codice coerente viene estratto e trasformato in un metodo separato per frammentare la logica originale.

Un'altra strategia frequente consiste nel riordinare i blocchi di codice indipendenti o nel cambiamento nella posizione di attributi e metodi all'interno delle classi. Poiché tali modifiche alterano profondamente l'ordine sequenziale delle istruzioni, esse richiedono strumenti di rilevazione capaci di costruire e confrontare rappresentazioni astratte, come gli alberi sintattici (AST) o i grafi delle chiamate (call-graphs), che sono intrinsecamente più robusti rispetto alla semplice disposizione di blocchi di testo.

2.3.3 Offuscamento semantico e refactoring avanzato

L'offuscamento semantico rappresenta la sfida più complessa per i sistemi di SCPD, poiché sfrutta la conoscenza del dominio per riscrivere la logica stessa dell'algoritmo. In questo ambito, il plagiante può optare per la sostituzione delle strutture dati, ad esempio rimpiazzando un array statico con una lista concatenata o una struttura dinamica equivalente, alterando così i pattern di accesso alla memoria ed i profili di esecuzione.

Queste tecniche di refactoring avanzato mirano a produrre versioni del software che, pur mantenendo lo stesso com-

portamento osservabile (input e output identici), appaiono radicalmente diverse davanti ad un'analisi statica. La letteratura evidenzia come la robustezza degli strumenti accademici o commerciali (come MOSS e JPLAG) debba essere costantemente testata contro tali trasformazioni. In particolare, con l'introduzione di strumenti come **COAT** (Code Obfuscation Tool) [7], progettato specificamente per generare varianti offuscate e valutare se i sistemi di rilevazione siano in grado di identificare la similarità semantica sottostante anche a fronte di cambiamenti logici significativi.

3 Tecniche e Algoritmi di Rilevamento (Approccio Teorico)

La rilevazione automatica del plagio nel codice sorgente pone una sfida complessa che va oltre il semplice confronto testuale: richiede la capacità di trasformare programmi informatici in rappresentazioni astratte che ne rendano confrontabile la logica, indipendentemente dallo stile di scrittura. La letteratura scientifica ha prodotto nel tempo una vasta gamma di algoritmi per rispondere a questa esigenza, i quali possono essere classificati efficacemente in base al livello di profondità con cui interpretano il codice sorgente.

3.1 Analisi Basata sugli Attributi (Attribute-based)

L'analisi basata sugli attributi, definita anche come approccio metrico, non esamina direttamente la struttura logica o il testo del codice, ma si focalizza sull'estrazione di un insieme di caratteristiche quantitative (software metrics) che descrivono il programma. L'assunto di base è che ogni autore possieda uno «stile di programmazione» unico, che si riflette in parametri statistici costanti.

3.1.1 Metriche software principali

In questo approccio, il codice sorgente viene trasformato in un vettore di attributi. Le metriche più comunemente utilizzate includono:

- **Metriche di Halstead:** Si basano sul conteggio degli operatori e degli operandi per misurare la complessità algoritmica, il volume e lo sforzo implementativo [8].
- **Complessità Ciclomatica di McCabe:** Misura il numero di percorsi linearmente indipendenti attraverso il grafo di flusso del controllo del programma, indicando quanto sia articolata la logica decisionale [9].
- **Indicatori di stile:** Conteggio delle righe di codice, numero di commenti, frequenza di determinate parole chiave del linguaggio, o la lunghezza media degli identificatori.

3.1.2 Meccanismo di confronto

Una volta estratte le metriche da due diversi file sorgente, la similarità viene calcolata determinando la «distanza» tra i due vettori risultanti (spesso utilizzando la distanza

euclidea o la similarità del coseno) [2]. Se la differenza tra i valori rientra in una soglia prestabilita, il sistema segnala un potenziale caso di plagio.

3.1.3 Analisi Critica: Robustezza e Limiti

Sebbene l'analisi basata sugli attributi offra vantaggi significativi in termini di efficienza computazionale e scalabilità su dataset di vaste dimensioni [3], la sua efficacia nella rilevazione di plagi sofisticati è severamente limitata. Il limite principale risiede nella natura stessa delle metriche software, le quali risultano estremamente sensibili a trasformazioni superficiali del codice [5].

L'inserimento di «codice morto» o di istruzioni irrilevanti altera i parametri quantitativi del programma, ingannando facilmente gli algoritmi di confronto basati sulle metriche. Poiché mancano di una reale comprensione della logica o della sintassi, questi approcci risultano vulnerabili a tecniche di offuscamento elementari. Di conseguenza, pur essendo utili per una scansione preliminare rapida, tali tecniche non sono considerate sufficientemente robuste per identificare casi di plagio strutturale o semantico.

3.2 Analisi Basata sul Testo (String-based)

L'analisi basata sul testo tratta il codice sorgente come una semplice sequenza di caratteri o stringhe. In questo approccio, il software viene confrontato a livello testuale per identificare segmenti di codice identici o simili tra due o più file.

3.2.1 Algoritmi e funzionamento

Le tecniche string-based si affidano ad algoritmi classici di *string-matching*. Tra i più rilevanti in letteratura si distinguono:

- **Longest Common Subsequence (LCS):** utilizzato per trovare la più lunga sottosequenza comune tra due stringhe, permettendo di identificare porzioni di codice che mantengono lo stesso ordine relativo [10].
- **Greedy String Tiling (GST):** un algoritmo che cerca di coprire un file con il maggior numero possibile di sottostringhe (o «tesseri») estratte da un altro file, risultando efficace anche se i blocchi di codice sono stati riordinati [11].

3.2.2 Efficacia e limitazioni

Questi metodi risultano particolarmente efficaci nel rilevamento del **plagio letterale (verbatim)**, ovvero la copia esatta di file in cui le uniche modifiche riguardano elementi non funzionali come spazi bianchi o commenti. Tuttavia, l'analisi testuale pura presenta vulnerabilità critiche come la sua sensibilità all'offuscamento lessicale. Gli approcci string-based sono messi in crisi da tecniche come lo *scrambling* degli identificatori e l'inserimento di «codice morto», poiché queste modifiche alterano la sequenza dei caratteri senza cambiare la logica.

Un altro problema di questo tipo di approcci riguarda le trasformazioni strutturali. Poichè questi strumenti non hanno una reale comprensione della sintassi, trasformazioni strutturali come il passaggio da un ciclo `for` a un ciclo `while` rendono il codice irriconoscibile per un confronto testuale puro.

3.2.3 Esempio di Plagio Letterale

Il codice originale e quello plagiato sono identici come sequenza di caratteri.

Codice originale:

```
int calculate_sum(int a, int b) {
    return a + b;
}
```

Codice plagiato rilevabile da string-based:

```
int calculate_sum(int a, int b) {
    return a + b;
}
```

Codice plagiato non rilevabile da string-based:

```
int sum(int x, int y) {
    return x+y;
}
```

Se il plagiatore cambiasse semplicemente i nomi delle variabili e/o delle funzioni di un software, un confronto basato puramente su stringhe potrebbe fallire o dare una similarità ridotta.

3.3 Analisi Basata sui Token (Token-based)

L'analisi basata sui token rappresenta un'evoluzione significativa rispetto all'analisi testuale pura. In questo approccio, il codice sorgente non viene trattato come una sequenza di caratteri, ma viene trasformato in una sequenza di **simboli lessicali** (token) attraverso un processo di analisi lessicale (**lexing**).

3.3.1 Tokenizzazione e Normalizzazione

Il vantaggio principale di questo metodo risiede nella sua capacità di astrarre il codice dalla sua forma superficiale. Il processo si articola generalmente in due fasi:

1. **Lexing:** Il codice sorgente viene scansionato e convertito in una stringa di token che rappresentano le unità logiche del linguaggio (es. `IF`, `WHILE`, `VAR`, `ASSIGN`).
2. **Normalizzazione:** Durante questa fase, vengono eliminate le informazioni irrilevanti dal punto di vista funzionale. In particolare vengono rimossi commenti e spazi bianchi, e gli identificatori (come nomi di variabili, funzioni o classi) vengono sostituiti da un token generico, in modo che il nome di una variabile non influisca sull'analisi.

Grazie a questa astrazione, il sistema è in grado di riconoscere la similarità anche se il plagiatore ha modificato radicalmente i nomi delle variabili (*scrambling*) o lo stile di formattazione.

3.3.2 Algoritmi di confronto

Una volta ottenuta la sequenza di token, il sistema applica algoritmi di confronto per identificare sottosequenze comuni. L'algoritmo più noto in questo ambito è il **Running Karp-Rabin Greedy String Tiling (RK-GST)**, utilizzato da strumenti come JPlag [1]. Questo algoritmo è particolarmente efficace in quanto è in grado di rilevare blocchi di codice che sono stati riordinati o spostati e utilizza tecniche di hashing (Karp-Rabin) per velocizzare il confronto su grandi volumi di dati, rendendolo più scalabile rispetto alla ricerca esaustiva.

Un altro approccio che vale la pena citare riguarda l'utilizzo di algoritmi di **winnowing**. Strumenti come MOSS [3] fanno uso di questo tipo di tecniche che, invece di confrontare l'intera sequenza, suddividono il codice in *k-grammi* (sottosequenze di lunghezza *k*). Di questi *k-grammi* viene calcolato l'hash e solo un sottoinsieme selezionato viene archiviato utilizzando una «finestra scorrevole».

3.3.3 Robustezza e limitazioni

L'analisi token-based è ampiamente considerata lo standard «de facto» per la maggior parte degli strumenti accademici grazie al suo equilibrio tra precisione e complessità computazionale. Tuttavia, come ogni metodologia di analisi statica, presenta specifici punti di forza e vulnerabilità critiche che ne determinano l'efficacia.

Il merito principale è la sua resistenza all'offuscamento lessicale. Poichè il processo di normalizzazione elimina commenti, spazi bianchi e rinomina gli identificatori, gli algoritmi basati su token risultano immuni a modifiche superficiali come lo *scrambling* delle variabili, o il cambiamento dello stile di formattazione.

3.3.4 Esempio di offuscamento lessicale

L'analisi basata sui token converte il codice in una sequenza di simboli astratti, ignorando i nomi delle variabili.

Codice A (originale):

```
int sumArray(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

Codice B (plagiato rilevabile da token-based):

```
int calcola(int[] valori) {
```

```
int x = 0;
for (int j = 0; k < valori.length; k++) {
    x += valori[k];
}
return x;
}
```

Codice C (plagiato *non* rilevabile da token-based):

```
int sumArray(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        int noise = 99;
        noise = noise * 2;
        total += arr[i];
    }
    return total;
}
```

Per comprendere il risultato del confronto, osserviamo come questi frammenti vengono «visti» dall'algoritmo dopo la fase di lexing e normalizzazione. In questa fase, commenti e spazi vengono rimossi e ogni identificatore (nome di variabile o funzione) viene sostituito da un token generico `ID`.

La sequenza generata per il **Codice A** ed il **Codice B** è identica poichè la struttura sintattica è la stessa, garantendo una rilevazione del 100%:

```
FOR ( ... ) { ID += ID [ ID ] ; }
```

Per il **Codice C** invece la sequenza viene disturbata dall'aggiunta di rumore all'interno del corpo del codice:

```
FOR ( ... ) { INT ID = LIT ; ID = ID * LIT
; ID += ID [ ID ] ; }
```

Come si evince dal confronto, mentre i codici A e B producono la stessa «impronta digitale», il codice C introduce nuovi statements, interrompendo la contiguità necessaria per la rilevazione del plagio.

3.4 Analisi Basata sulla Struttura (Tree-based)

L'analisi basata sulla struttura rappresenta un approccio più sofisticato rispetto all'analisi lessicale (token-based), in quanto non si limita a convertire il codice in una sequenza lineare di simboli, ma ne astrae la struttura sintattica e logica completa. In questo metodo, il codice sorgente viene analizzato (parsed) per costruire una *Intermediate Representation* (IR) tipicamente sotto la forma di un *Abstract Syntax Tree* (AST) o, in alcuni casi, di un *Program Dependency Graph* (PDG).

3.4.1 Costruzione dell'AST e Astrazione

Il processo inizia con il parsing del codice sorgente, che trasforma il testo in una struttura ad albero dove ogni nodo rappresenta un costrutto sintattico (es. cicli, assegnazioni, espressioni condizionali). A differenza dei metodi *string-based* o *token-based*, l'approccio *tree-based* elimina intrinsecamente le differenze superficiali: la struttura dell'albero rimane identica indipendentemente dai nomi delle variabili, dai commenti o dalla formattazione del codice [3], [12]. Ad esempio, l'AST cattura la gerarchia delle operazioni e l'annidamento dei blocchi logici, offrendo una «impronta digitale» strutturale del programma.

3.4.2 Algoritmi di confronto

Una volta generati gli AST per i programmi da confrontare, la rilevazione del plagio si riduce ad un problema di *tree matching* o di ricerca di sotto-alberi isomorfi¹. Gli algoritmi cercano di identificare sotto-strutture simili o identiche tra due AST.

Si parla di **confronto esatto** quando la verifica richiede che i due alberi siano identici. È efficace solo per copie quasi esatte. Si parla invece di **confronto approssimato** quando vengono utilizzate metriche di similarità per quantificare quanto due alberi si sovrappongono. Questo è necessario poichè piccole modifiche al codice (come l'inserimento di un'istruzione vuota) possono alterare la struttura dell'albero.

3.4.3 Vantaggi e limitazioni

L'approccio Tree-based rileva il Plagio di Tipo 3 (piccole aggiunte/rimozioni di codice) meglio dei token, poichè un ramo aggiunto all'albero non invalida la struttura dei rami adiacenti. Il vantaggio principale di questo approccio è appunto la sua robustezza contro l'offuscamento lessicale e il riordinamento di funzioni o metodi, poichè la struttura gerarchica (ad esempio della classe) viene analizzata indipendentemente dall'ordine testuale. Tuttavia, l'analisi *tree-based* presenta svantaggi in termini di complessità computazionale. Il confronto di alberi è un'operazione costosa (di complessità polinomiale o superiore), rendendo questi strumenti meno scalabili su grandi repository rispetto ai metodi *token-based*. Inoltre, modifiche strutturali significative, come la sostituzione di un ciclo `for` con un ciclo `while`, modificano la forma dell'albero, rendendo il plagio più difficile da rilevare se l'algoritmo di confronto è troppo rigido.

Per superare i limiti strutturali, ricerche recenti hanno proposto di arricchire la classica rappresentazione ad albero integrando esplicitamente archi supplementari che tracciano il flusso di controllo e dei dati. Questa rappresentazione ibrida prende il nome di Flow-Augmented

¹Due sotto-alberi T_1 e T_2 si dicono isomorfi se esiste una corrispondenza biunivoca tra i loro nodi tale che la struttura di adiacenza sia preservata. In termini pratici, due alberi sono isomorfi se possono essere sovrapposti perfettamente tramite una rotazione o un riordinamento dei figli, mantenendo invariate le connessioni gerarchiche.

AST [12] e permette di mantenere la gerarchia sintattica dell'albero incorporando le dipendenze semantiche necessarie a rilevare manipolazioni complesse del codice.

3.4.4 Esempio di Plagio Strutturale

L'analisi basata sulla struttura (AST) astrae il codice in una gerarchia logica. Questo la rende resistente all'inserimento di blocchi di codice superflui che alterano la sequenza lineare dei token, ma rimane vulnerabile a cambiamenti radicali dell'algoritmo.

Codice A (Originale):

```
int sumArray(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

Codice B (Plagiato rilevabile da Tree-based):

```
int arraySum(int[] values) {
    {
        int dummy = 0;
        int result = 0;
        for (int k = 0; k < values.length; k++) {
            result += values[k]
        }
        return values;
    }
}
```

Codice C (Plagiato non rilevabile da tree-based):

```
int sumArray(int[] arr) {
    int total = 0;
    int i = 0;
    while(i < arr.length) {
        total += arr[i];
        i++;
    }
    return total;
}
```

Per comprendere la differenza rispetto all'analisi token-based, osserviamo come questi frammenti vengono rappresentati gerarchicamente dall'albero sintattico.

Nel confronto tra **Codice A** e **Codice B**, l'algoritmo di tree matching è in grado di rilevare la similarità poiché cerca sotto-alberi isomorfi.

- **AST Codice A:** Il nodo radice del metodo contiene direttamente il nodo `ForStatement`.
- **AST Codice B:** Il nodo radice contiene un nodo `Block` intermedio, che a sua volta contiene il nodo `ForStatement`.

L'algoritmo ignora il «contenitore» extra e identifica che il sotto-albero che parte da `ForStatement` è strutturalmente identico (ha gli stessi figli: inizializzazione, condizione, incremento e corpo) in entrambi i casi.

Nel confronto con il **Codice C**, invece, la topologia dell'albero cambia, in quanto il nodo `ForStatement` viene sostituito dal nodo `WhileStatement` e l'istruzione di inizializzazione `int i = 0` che era prima incapsulata nel ciclo viene ora spostata fuori.

Poiché i nodi non corrispondono più né per tipo né per posizione gerarchica, la distanza di edit tra gli alberi (Tree Edit Distance) risulta elevata, portando a una mancata rilevazione del plagio.

3.5 Analisi Basata sul Grafo (Graph-based)

L'analisi basata sul grafo rappresenta un ulteriore passo verso la comprensione semantica del codice. Mentre l'AST analizza la gerarchia sintattica (com'è scritto il codice), l'approccio *graph-based* modella il flusso di esecuzione e le dipendenze tra i dati (cosa fa il codice).

La rappresentazione più utilizzata in questo ambito è il **Program Dependency Graph** (PDG). In un PDG, i **nodi** rappresentano le istruzioni o le espressioni e gli **archi** rappresentano due tipi di relazioni:

1. **Dipendenza dai dati:** L'istruzione *B* usa una variabile definita in *A*.
2. **Dipendenza dal controllo:** L'istruzione *B* viene eseguita solo se la condizione in *A* è vera

3.5.1 Superare i limiti della sintassi

Il vantaggio cruciale del PDG è che astrae completamente l'ordine testuale delle istruzioni che non sono dipendenti tra loro, e soprattutto, unifica costrutti sintattici diversi che hanno lo stesso comportamento. Se un ciclo `for` e un ciclo `while` generano AST diversi nell'analisi *tree-based*, poiché in realtà entrambi implementano la stessa logica di iterazione, generano PDG pressoché identici. Le dipendenze rimangono invariate, permettendo all'algoritmo di rilevare plagio anche su cloni di tipo 4/4².

²Il Tipo 4 (Semantic Clone) si verifica quando due frammenti di codice eseguono la stessa computazione (stesso comportamento funzionale) ma sono implementati attraverso varianti sintattiche differenti. Esempi tipici includono la sostituzione di un ciclo `for` con un `while` o l'uso della ricorsione al posto dell'iterazione [13]

3.5.2 Algoritmi di confronto: Isomorfismo di Sottografi

Una volta costruiti i PDG dei programmi da analizzare, il rilevamento del plagio si riduce a un problema matematico noto come Isomorfismo di Sottografi (Subgraph Isomorphism). Formalmente, dato un grafo «pattern» G_P (il codice sospetto o una sua parte) e un grafo «target» G_T (il codice originale), l'obiettivo è determinare se esiste una mappatura biunivoca che permetta di sovrapporre G_P a una porzione di G_T , preservando la struttura delle adiacenze e i tipi di dipendenza (dati o controllo).

Il limite principale di questo approccio è la sua intrattabilità teorica: il problema dell'isomorfismo di sottografi appartiene alla classe di complessità NP-Hard³. Ciò significa che, nel caso peggiore, il tempo necessario per il confronto cresce esponenzialmente con il numero di nodi del grafo. Applicare un algoritmo esatto di graph matching su repository contenenti migliaia di file sorgente risulterebbe computazionalmente proibitivo.

Per rendere questa tecnica applicabile in scenari reali, gli strumenti dello stato dell'arte come GPLAG [14] adottano strategie di approssimazione e filtraggio. Ad esempio, prima di avviare il costoso confronto sui grafi, viene eseguito un pre-screening rapido basato su metriche semplici (come il numero di nodi o la distribuzione dei tipi di istruzioni). Se due grafi sono troppo diversi «macroscopicamente», vengono scartati immediatamente.

3.5.3 Esempio di Plagio semantico

Come detto precedentemente il PDG non si basa solamente sulla struttura sintattica del programma, ma riesce a creare un'astrazione delle dipendenze per poter analizzare il flusso di esecuzione senza dover necessariamente eseguire il programma. Il rilevatore fallisce nel momento in cui l'algoritmo viene effettivamente cambiato.

Codice A (Originale):

```
int sumArray(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i]
    }
    return total;
}
```

Codice B (Plagiato rilevabile da Graph-based):

```
int arraySum(int[] values) {
    int result = 0;
    int j = 0;
    while (j < values.length) {
```

```
        result += values[j];
        j++;
    }
    return result;
}
```

Codice C (Plagiato non rilevabile da Graph-based):

```
int sumArray (int[] arr) {
    return Arrays.stream(arr).sum();
}
```

Osservando i frammenti proposti, emerge chiaramente la capacità del Grafo delle Dipendenze del Programma di superare le barriere sintattiche che limitavano gli approcci precedenti. Nel confronto tra il **Codice A** e il **Codice B**, sebbene la struttura testuale sia stata alterata sostituendo il ciclo `for` con un costrutto `while` e dislocando le istruzioni di inizializzazione e incremento, il grafo risultante è isomorfo. In entrambi i casi, infatti, l'algoritmo rileva le medesime relazioni di dipendenza: la variabile accumulatore dipende dal valore estratto dall'array, il quale a sua volta è vincolato all'indice che governa la condizione di uscita del ciclo.

Al contrario, il Codice C introduce una variazione paradigmatica che rende il plagio invisibile a questa tecnica. L'utilizzo delle API funzionali (Stream) rimuove esplicitamente le variabili di controllo e i cicli iterativi, generando un grafo delle dipendenze lineare o monolitico che non presenta alcuna somiglianza strutturale con il grafo ciclico generato dall'algoritmo originale.

3.6 Approcci basati su Machine Learning e Deep Learning (Learning-based)

Le tecniche analizzate nei paragrafi precedenti (String, Token, Tree, Graph) condividono una caratteristica comune: si basano su regole e algoritmi deterministici definiti a priori dall'uomo. Sebbene efficaci, questi metodi faticano a scalare verso la comprensione semantica profonda, specialmente quando due codici svolgono la stessa funzione usando paradigmi completamente diversi.

Negli ultimi anni, la ricerca si è spostata verso approcci *learning-based*, sfruttando il Machine Learning e, più recentemente, il Deep Learning per addestrare modelli capaci di classificare la similarità del codice automaticamente.

3.6.1 Rappresentazione Vettoriale

Il cuore di questa rivoluzione risiede nel superamento dell'ingegneria manuale delle feature. Invece di contare metriche o confrontare alberi, le moderne architetture

³Un problema è NP-Hard se ogni problema appartenente alla classe NP è ad esso riducibile in tempo polinomiale. Di conseguenza, non è noto alcun algoritmo che lo risolva in tempo polinomiale nel caso peggiore, rendendo la ricerca di soluzioni esatte computazionalmente intrattabile per istanze di grandi dimensioni.

di Deep Learning (come le Graph Neural Networks o i modelli basati su Transformer come CodeBERT [15]) trasformano il codice sorgente in **Code Embeddings**.

Un embedding è una rappresentazione del codice sotto forma di vettore numerico denso (un array di numeri reali) in uno spazio multidimensionale. Il modello viene addestrato affinché frammenti di codice semanticamente simili vengano mappati in punti vicini nello spazio vettoriale, mentre codici con funzionalità diverse vengano proiettati in punti distanti. La rilevazione del plagio diventa quindi un calcolo geometrico: si misura ad esempio la Similarità del Coseno tra i vettori dei due programmi. Se l'angolo tra i vettori è sufficientemente ridotto, i programmi sono considerati cloni, indipendentemente dalla loro struttura sintattica superficiale.

3.6.2 Interpretabilità dei risultati

Tuttavia, questa straordinaria capacità di astrazione comporta un costo significativo in termini di trasparenza, noto come il problema della **Black Box**. Mentre strumenti come JPlag (token-based) offrono una prova visiva immediata, evidenziando le righe di codice copiate e mostrando la corrispondenza esatta, un modello di Deep Learning restituisce unicamente un valore di probabilità (es. «Similarità: 98%»).

Poiché il vettore è il risultato di milioni di operazioni matriciali non lineari all'interno della rete neurale, è estremamente difficile, se non impossibile, risalire a quali specifiche istruzioni abbiano determinato il verdetto di plagio. Questa mancanza di Explainability rappresenta un ostacolo critico in ambito accademico e legale, dove l'accusa di plagio deve essere supportata da prove tangibili e interpretabili, e non solo da un punteggio statistico.

Approcci avanzati basati su Graph Neural Networks (GNN) stanno però iniziando a mitigare il problema della Black Box [12]. Applicando meccanismi di attenzione (attention mechanisms) alle rappresentazioni vettoriali del codice, è possibile evidenziare quali specifici nodi o sottostrutture del grafo abbiano contribuito maggiormente al calcolo della similarità. Questo non solo migliora l'accuratezza su plagi semantici, ma offre un primo livello di interpretabilità visiva, permettendo di identificare le porzioni di codice sospette anche in assenza di corrispondenze testuali esatte.

3.6.3 Esempio di Plagio con Variazione di Paradigma

Nonostante i limiti di spiegabilità, il Deep Learning è l'unica arma efficace contro i plagi semantici complessi. Riprendiamo l'esempio che aveva messo in crisi l'analisi Graph-based (PDG) nel capitolo precedente.

Codice A (originale):

```
int sumArray(int[] arr) {
```

```
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

Codice C (paradigma funzionale):

```
int sumArray(int[] arr) {
    return Arrays.stream(arr).sum();
}
```

Un modello di Deep Learning pre-addestrato su milioni di repository open-source ha «imparato» che il pattern `Arrays.stream(...).sum()` appare frequentemente negli stessi contesti del pattern `for (...) { ... += ... }`. Di conseguenza, il modello genererà due vettori numerici molto simili per il **Codice A** e il **Codice C**, rilevando il plagio semantico che era sfuggito a tutte le tecniche precedenti.

3.7 Approcci LLM-Based

I Large Language Models (LLM) rappresentano una categoria qualitativamente nuova rispetto a tutti gli approcci discussi in precedenza. A differenza dei modelli learning-based classici, un LLM non viene addestrato su coppie di codice etichettate come «plagio/non plagio»: la sua comprensione del codice emerge dalla pre-esposizione a miliardi di parametri addestrati su corpus enormi che includono codice sorgente, documentazione tecnica e linguaggio naturale. Il task di SCPD viene quindi affrontato in modalità zero-shot (senza nessun addestramento specifico sul problema), mediante la costruzione di un prompt in linguaggio naturale che descrive il problema.

3.7.1 Meccanismo di funzionamento

L'interfaccia con un LLM non avviene attraverso una pipeline deterministica come nei metodi precedenti, ma tramite un prompt: un testo in linguaggio naturale che descrive il task, fornisce i frammenti di codice da confrontare e specifica il formato atteso della risposta. La qualità del prompt è determinante per l'affidabilità del risultato — un prompt vago produce verdetti inconsistenti, mentre un prompt strutturato e dettagliato, che specifica esplicitamente la tassonomia dei tipi di clone e richiede una motivazione, tende a produrre analisi più precise e riproducibili. La risposta del modello può essere ottenuta in linguaggio naturale — dove il modello argomenta liberamente le somiglianze rilevate — oppure in un formato strutturato come JSON, specificandolo nel prompt, in modo da rendere l'output comparabile con i punteggi numerici degli altri strumenti. Questa flessibilità è una caratteristica distintiva dell'approccio: lo stesso modello può

adattarsi a contesti diversi semplicemente modificando il prompt, senza alcuna modifica al sistema sottostante.

3.7.2 Evidenze Empiriche

Studi recenti hanno iniziato a valutare sistematicamente le capacità degli LLM come strumenti di SCPD, confrontandoli con i tool tradizionali su dataset che coprono tutti e quattro i tipi di plagio. I risultati mostrano che GPT-4o raggiunge un'accuratezza complessiva del 78,70% con F1 score di 86,97%, mentre LLaMA 3 — modello open source e self-hostabile — ottiene un'accuratezza del 71,53% con F1 score di 82,75% [16]. Entrambi i modelli dimostrano capacità superiori agli strumenti token-based nel rilevamento dei cloni di Tipo 4, ovvero le variazioni semantiche profonde che JPlag e MOSS non riescono a catturare.

3.7.3 Vantaggi e Limiti

Il vantaggio principale rispetto a tutti gli approcci precedenti è la spiegabilità nativa: invece di un punteggio numerico opaco, il modello produce un ragionamento in linguaggio naturale che identifica esplicitamente quali costrutti, logiche o pattern siano stati replicati. Questo supera il problema della Black Box discusso nella sezione 3.6.2 e rende i risultati più direttamente utilizzabili in un contesto accademico o legale.

Un secondo vantaggio è il supporto cross-language: un LLM pre-addestrato su più linguaggi simultaneamente può confrontare un frammento Python con uno Java, rilevando somiglianze semantiche che nessuno strumento monolingua potrebbe identificare. Questa capacità è particolarmente rilevante nel contesto delle nuove sfide discusse nella Sezione 6.

Tuttavia l'approccio presenta limiti strutturali che ne condizionano l'adozione su scala. Il primo è il costo computazionale: confrontare un corpus di centinaia di submission richiede un numero elevato di chiamate API o l'esecuzione locale di modelli da miliardi di parametri, rendendo l'approccio significativamente più lento rispetto a JPlag o MOSS. Il secondo è il non determinismo: lo stesso prompt eseguito due volte può produrre verdetti leggermente diversi, rendendo difficile la piena riproducibilità degli esperimenti. Il terzo è il rischio di falsi positivi: gli LLM tendono a segnalare come plagio frammenti che condividono pattern comuni di implementazione — come algoritmi di ordinamento standard — senza che vi sia un effettivo riutilizzo non autorizzato.

3.7.4 Esempio di Plagio con variazione di Paradigma

Per illustrare la capacità distintiva dell'approccio LLM-based, consideriamo una variante dell'esempio utilizzato nelle sezioni precedenti in cui non solo il paradigma ma anche il linguaggio di programmazione cambia tra i due frammenti.

Codice A (Originale — Java):

```
int sumArray(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

Codice B (Plagiato — Python, paradigma funzionale):

```
def sum_array(arr):
    return sum(arr)
```

Qualsiasi strumento deterministico — string-based, token-based, tree-based o graph-based — produrrebbe una similarità prossima allo zero: i due frammenti non condividono alcun token, nessuna struttura sintattica, nessuna dipendenza confrontabile. Persino un modello DL fine-tuned su Java faticerebbe a generalizzare su questa coppia, in quanto i due frammenti appartengono a spazi vettoriali addestrati su distribuzioni diverse.

Un LLM interrogato con un prompt adeguato riconosce invece che entrambi i frammenti implementano la stessa operazione — la somma degli elementi di una sequenza — ragionando sul comportamento osservabile piuttosto che sulla forma. La risposta del modello potrebbe argomentare che la funzione built-in `sum()` di Python è semanticamente equivalente al pattern di accumulazione con ciclo `for` in Java, identificando il plagio cross-language nonostante la completa diversità sintattica, strutturale e linguistica. Questa capacità, unica tra tutti gli approcci analizzati, è al contempo il punto di forza più rilevante e la fonte principale del rischio di falsi positivi discusso in 3.7.3: la somma di un array è un pattern talmente comune che due implementazioni indipendenti potrebbero essere erroneamente classificate come plagio.

3.8 Sintesi Comparativa

A conclusione delle tecniche teoriche di rilevazione del plagio del codice sorgente, la seguente tabella riassume le capacità ed i limiti delle diverse metodologie discusse.

Tecnica	Astrazio- ne	Tipo di Clone	Resistenza Offuscamen- to	Spiegabi- lità
string- based	Testo	Tipo 1	Bassa	Altissima
token- based	Lessicale	Tipo 1, 2	Media	Alta
tree-ba- sed	Sintattica	Tipo 1, 2, 3	Alta	Media
graph- based	Semanti- ca	Tipo 1-3,4 (Parziale)	Molto Alta	Bassa
learning- based	Vettoriale	Tutti	Massima	Nulla (Black Box)
LLM-ba- sed	Linguisti- ca	Tutti + cross-lan- guage	Massima	Altissima

Tabella 1: Confronto tra le principali metodologie di rilevazione del plagio del codice sorgente

4 Rassegna degli Strumenti (State of the Art)

In questa sezione verranno analizzati i principali strumenti di rilevazione del plagio del codice sorgente (SCDP) sviluppati nel corso degli anni. La rassegna si concentra sulle soluzioni accademiche e open source che hanno definito lo standard del settore, per poi esplorare piattaforme commerciali più recenti. Ogni strumento verrà valutato in base alle tecniche algoritmiche adottate (discusse nel Capitolo 3), ai linguaggi supportati e alla robustezza contro le tecniche di offuscamento.

4.1 Strumenti Accademici e Open Source

Il panorama degli strumenti accademici è dominato da soluzioni storiche che, nonostante l'età, rimangono i punti di riferimento per l'analisi statica del plagio. Questi tool sono tipicamente gratuiti per uso educativo e si basano su analisi testuale o token-based

4.1.1 JPlag⁴

Sviluppato originariamente presso l'Università di Karlsruhe (Germania) da Prechelt, Malpohl e Philippsen e descritto formalmente nel 2002 [1], JPlag è considerato il principale strumento accademico open source per la rilevazione del plagio. Creato nel 1996, è ancora oggi attivamente sviluppato e mantenuto presso il Karlsruhe Institute of Technology (KIT), e supporta Java, C, C++, C#, Python, JavaScript, TypeScript, Go, Rust, Kotlin, Swift, Scala e altri linguaggi.

Internamente, JPlag opera in due fasi principali: prima effettua il parsing del programma, estraendo un sottoinsieme dei nodi del parse tree sotto forma di token, lineariz-

zando così la struttura ad albero; successivamente applica tecniche di normalizzazione per aumentare la resilienza contro l'offuscamento [17]. Per il confronto delle sequenze, JPlag utilizza l'algoritmo **Running Karp-Rabin Greedy String Tiling (RK-GST)**, che cerca di coprire la sequenza di token di un programma con le sottosequenze contigue più lunghe trovate nell'altro, indipendentemente dalla loro posizione originale.

Un aspetto cruciale che distingue JPlag dagli altri strumenti storici è il suo sviluppo continuo in risposta alle minacce emergenti. La diffusione di tecniche di offuscamento automatico mette in discussione l'assunzione che eludere un rilevatore di plagio richieda più sforzo del completare un compito di programmazione, minacciando i rilevatori privi di resilienza e, in ultima analisi, l'integrità accademica [17]. Per rispondere a questa sfida, il team di KIT ha introdotto due meccanismi di difesa complementari: la Token Sequence Normalization (TSN), progettata per neutralizzare attacchi basati sull'inserimento di codice morto e sul riordinamento di istruzioni, e il Subsequence Match Merging (SMM), una tecnica euristica che contrasta attacchi volti a interrompere la contiguità delle corrispondenze tra sequenze di token. I risultati dimostrano che JPlag supera significativamente gli strumenti concorrenti in termini di resilienza: nei benchmark contro tecniche di offuscamento automatico, le istanze di plagio raggiungono una similarità mediana del 100%, contro il 26.7% di Dolos e il 7.5% di MOSS [17].

JPlag è open source e può essere eseguito localmente, risultando conforme al GDPR — un aspetto rilevante per i contesti accademici europei che trattano dati sensibili degli studenti. L'interfaccia grafica include una vista di distribuzione delle similarità per identificare outlier sospetti, una vista di confronto dettagliata per analizzare i segmenti di codice corrispondenti, e una cluster view che aiuta a comprendere dinamiche di plagio collettivo tra submission multiple.

Nonostante i continui progressi, l'alterazione semantica profonda e il cambio radicale di paradigma algoritmico (cloni di Tipo 4) restano un limite strutturale intrinseco dell'approccio token-based. Inoltre, i meccanismi di difesa attuali non coprono ancora in modo efficace il plagio generato interamente da AI sulla base della descrizione del compito, una minaccia distinta dall'offuscamento classico e ancora aperta nella letteratura.

4.1.2 SIM⁵

Sviluppato originariamente da Dick Grune [18] alla fine degli anni '80 e formalizzato in successivi studi, SIM rappresenta uno degli strumenti più longevi nell'ambito

⁴<https://github.com/jplag/JPlag/>

⁵https://dickgrune.com/Programs/similarity_tester/

della rilevazione del plagio. Nonostante l'età, rimane un punto di riferimento frequente negli studi comparativi.

Lo strumento supporta diversi linguaggi di programmazione, tra cui C, C++, Java, Pascal ed il linguaggio naturale. Anche SIM lavora principalmente con la tokenizzazione e attraverso l'utilizzo di *flex* fornisce degli analizzatori lessicali dedicati ai diversi linguaggi [19]. In questa fase gli elementi irrilevanti come spazi e commenti vengono rimossi e le istruzioni vengono convertite in una sequenza lineare di token. Successivamente SIM applica un algoritmo di allineamento di stringhe (basato sulla ricerca della Longest Common Subsequence) per confrontare i flussi di token generati, individuando le sequenze contigue più lunghe condivise tra i due file.

Dal punto di vista prestazionale, l'implementazione dell'allineamento rende SIM computazionalmente molto efficiente e altamente preciso nell'identificare plagi di *Tipo 1* e *Tipo 2* (come il renaming delle variabili). Tuttavia, l'efficacia dell'algoritmo dipende dal mantenimento dell'ordine globale delle istruzioni: tecniche di offuscamento strutturale, come lo spostamento di blocchi di codice indipendenti, l'inversione di funzioni o l'inserimento di codice morto, spezzano le sequenze contigue e causano un drastico calo della similarità rilevata.

4.1.3 Plaggie

Plaggie è uno strumento open source sviluppato presso l'Helsinki University of Technology nel 2006 [20], concepito come alternativa libera e autonoma a MOSS per l'uso accademico. A differenza di MOSS, che opera come servizio remoto richiedendo l'invio del codice a server di Stanford, Plaggie è distribuito come applicazione Java standalone eseguibile localmente, il che lo rende particolarmente adatto a contesti in cui la privacy dei dati degli studenti è una preoccupazione rilevante.

L'architettura di Plaggie segue un approccio rigorosamente token-based, concettualmente analogo a quello di JPlag. Il processo si articola in due fasi principali: nella prima, il codice sorgente viene analizzato da un lexer specifico per il linguaggio che converte il testo in una sequenza lineare di token astratti, eliminando commenti, spazi bianchi e normalizzando gli identificatori. Nella seconda fase, le sequenze di token vengono confrontate tramite l'algoritmo Greedy String Tiling (GST), che individua le sottostringhe contigue più lunghe comuni tra coppie di programmi, indipendentemente dalla loro posizione nel file originale.

Una caratteristica distintiva di Plaggie rispetto a JPlag è la sua focalizzazione quasi esclusiva su Java come linguaggio target, scelta che ha permesso agli autori di ottimizzare il lexer per la struttura sintattica del linguaggio, migliorando la qualità della tokenizzazione per costrutti tipici

della programmazione orientata agli oggetti (classi, interfacce, generics).

In termini di robustezza, Plaggie condivide i punti di forza e i limiti strutturali dell'approccio token-based descritto nella sezione 3.3. Risulta efficace contro l'offuscamento lessicale (renaming di variabili e metodi) e contro semplici riordinamenti di istruzioni indipendenti, poiché il processo di normalizzazione rende queste trasformazioni trasparenti all'algoritmo. Tuttavia, come tutti i sistemi basati su GST puro, rimane vulnerabile all'inserimento massiccio di codice «rumore» che spezza la contiguità delle sequenze di token, e non è in grado di rilevare cloni di Tipo 4 (variazioni semantiche profonde o cambi di paradigma algoritmico).

Dal punto di vista della scalabilità, la complessità dell'algoritmo GST — superiore a quella del Winnowing utilizzato da MOSS — rende Plaggie meno performante su corpus di grandi dimensioni. Lo strumento è stato concepito per l'analisi di singoli corsi universitari (decine o centinaia di submission), non per scansioni su repository di scala industriale.

Sebbene Plaggie non abbia avuto lo stesso sviluppo continuo di JPlag e non sia più attivamente mantenuto, mantiene una presenza negli studi comparativi della letteratura come punto di riferimento storico per la valutazione di strumenti token-based su dataset Java. La sua semplicità architetturale e la disponibilità del codice sorgente lo rendono uno strumento utile per studi di benchmark e per contesti didattici in cui si desidera analizzare il funzionamento interno di un rilevatore di plagio.

4.1.4 Dolos⁶

Dolos è un progetto di ricerca attivo sviluppato dal Team Dodona dell'università di Gent (Belgio), con la prima pubblicazione scientifica di riferimento risalente al 2022. Il codice sorgente è rilasciato sotto licenza MIT, rendendolo liberamente utilizzabile e modificabile. Rispetto agli strumenti storici come MOSS e JPlag, Dolos nasce con una filosofia esplicitamente orientata all'usabilità: la complessità di strumenti come JPlag e MOSS ostacola spesso la loro adozione diffusa [21], e Dolos si propone di abbassare questa barriera.

L'elemento tecnico più distintivo di Dolos rispetto agli altri strumenti token-based è l'utilizzo di tree-sitter come motore di parsing. Tree-sitter è una libreria di parsing incrementale che genera AST per decine di linguaggi di programmazione attraverso modelli generici, rendendo Dolos language-agnostic [22]: aggiungere il supporto a un nuovo linguaggio non richiede la scrittura di un lexer dedicato, ma semplicemente l'integrazione del relativo modello tree-sitter. Una volta costruito l'AST, Dolos estrae una sequenza di token dalla visita dell'albero e applica

⁶<https://dolos.ugent.be>

l'algoritmo di Winnowing per il calcolo delle fingerprint, in modo concettualmente analogo a MOSS ma con il vantaggio di operare su token derivati dalla struttura sintattica piuttosto che sul testo grezzo.

Una caratteristica che distingue nettamente Dolos dagli altri strumenti open source è l'attenzione all'esperienza utente. L'ecosistema Dolos include una web app self-hostable, un'API JSON, una CLI, una libreria JavaScript e un container Docker preconfigurato [23]. Le dashboard analitiche offrono visualizzazioni interattive che permettono di identificare immediatamente i casi sospetti e, soprattutto, di rilevare fenomeni di plagio di gruppo attraverso un sistema di clustering delle submission: una funzionalità assente negli strumenti più datati, dove l'analisi è sempre e solo su coppie di file.

Piattaforme come Codio⁷ hanno abbandonato MOSS e JPlag in favore di una istanza self-hosted di Dolos [23], citando esplicitamente la semplicità di integrazione e la qualità dei report come motivazioni principali. Sul piano delle prestazioni, i benchmark sul dataset SOCO mostrano che Dolos supera gli strumenti dei concorrenti nella rilevazione di casi di plagio [22].

Come tutti gli approcci basati su fingerprinting e analisi token/AST, Dolos rimane vulnerabile alle trasformazioni semantiche profonde (cloni di Tipo 4) e al plagio generato da strumenti di AI generativa. Inoltre, pur essendo self-hostable, l'istanza pubblica ospitata da Ghent University garantisce una retention dei dati di soli 30 giorni, il che può rappresentare un limite per contesti che richiedono un archivio storico delle analisi.

4.1.5 Oreo⁸

Oreo è uno strumento open source per la rilevazione di cloni nel codice sorgente che adotta un approccio ibrido, combinando metriche software, information retrieval e machine learning in una pipeline sequenziale [24]. A differenza degli strumenti token-based e tree-based discussi nelle sezioni precedenti, Oreo non si limita ad analizzare la forma sintattica del codice, ma introduce un livello di comprensione semantica attraverso un modello di deep learning addestrato su coppie di frammenti, avvicinandosi alla categoria learning-based descritta nella Sezione 3.6.

L'obiettivo dichiarato di Oreo è superare il limite comune a quasi tutti gli strumenti discussi finora: la cosiddetta *Twilight Zone*, ovvero quello spettro di cloni che, pur mantenendo ancora qualche similarità sintattica, risultano estremamente difficili da rilevare — collocandosi al confine tra il Tipo 3 e il Tipo 4. Questa zona grigia rappresenta esattamente quei casi di plagio strutturale avanzato che i

metodi token-based e tree-based non riescono a gestire in modo affidabile.

Per affrontare questo problema, Oreo introduce un filtro semantico denominato Action filter che scarta un gran numero di coppie di codice prive di similarità semantica, risolvendo il problema dell'esplosione combinatoria tipico dei confronti a coppie su grandi repository. Le coppie che superano questo filtro vengono poi validate da un modello di deep learning che ne verifica la similarità strutturale, aumentando la precisione complessiva della rilevazione. L'approccio è quindi ibrido per costruzione: combina metriche software, information retrieval e machine learning in una pipeline sequenziale, dove ogni fase riduce lo spazio di ricerca per la fase successiva.

La valutazione della recall è stata condotta su BigCloneBench⁹, mentre la precision è stata stimata tramite ispezione manuale. I risultati dimostrano che Oreo raggiunge valori elevati su entrambe le metriche, con una capacità di rilevazione che si estende significativamente oltre i cloni con similarità sintattica moderata o debole, mantenendo al contempo buone proprietà di scalabilità.

Dal punto di vista pratico, per eseguire Oreo è necessario generare un file di input attraverso il Metric Calculator, uno strumento fornito insieme al sistema che scansiona le directory di progetto cercando file .java e ne calcola le metriche dei metodi. Questo rappresenta il limite più critico di Oreo in relazione agli obiettivi del presente studio: il supporto è esclusivamente per Java, il che esclude qualsiasi confronto diretto con strumenti multi-linguaggio come JPlag o MOSS su dataset eterogenei. Nei test sperimentali della Sezione 5, questo vincolo sarà discusso nella scelta del dataset e del linguaggio di riferimento.

4.1.6 CodeBERT/GraphCodeBERT

CodeBERT e GraphCodeBERT sono modelli Transformer pre-addestrati sviluppati da Microsoft Research per la comprensione del codice sorgente. A differenza di tutti gli strumenti discussi finora, non si tratta di tool autonomi con un'interfaccia di input/output pronta all'uso: sono modelli linguistici i cui pesi sono rilasciati pubblicamente su HuggingFace¹⁰ (`microsoft/codebert-base`, `microsoft/graphcodebert-base`) e che richiedono la costruzione di una pipeline sperimentale per essere applicati al task della SCPD.

CodeBERT è un modello multi-linguaggio pre-addestrato su coppie codice/linguaggio naturale in sei linguaggi di programmazione — Python, Java, JavaScript, PHP, Ruby e Go — e supporta task downstream come la ricerca di codice [15], la generazione di documentazione e la clone

⁷<https://www.codio.com>

⁸<https://github.com/Mondego/oreo>

⁹<https://github.com/clonebench/BigCloneBench>

¹⁰<https://huggingface.co>

detection. Per quest'ultimo task, il modello viene sottoposto a fine-tuning su un dataset etichettato di coppie di frammenti — tipicamente BigCloneBench — in una configurazione di classificazione binaria: la coppia viene passata sequenzialmente attraverso CodeBERT come encoder, i due vettori risultanti vengono concatenati e passati a un classificatore shallow per produrre il verdetto finale.

GraphCodeBERT rappresenta l'evoluzione diretta di CodeBERT. Mentre CodeBERT opera su coppie codice/linguaggio naturale sfruttando principalmente informazioni lessicali e sintattiche, GraphCodeBERT incorpora esplicitamente la struttura del grafo del flusso di dati durante il pre-addestramento, catturando le relazioni semantiche tra le variabili del programma [25]. Microsoft fornisce una pipeline ufficiale per il fine-tuning di GraphCodeBERT sul task di clone detection, basata sul dataset BigCloneBench e configurabile tramite script Python. Questa pipeline costituisce il punto di partenza per l'utilizzo sperimentale del modello nel presente studio.

Il vantaggio principale di entrambi i modelli rispetto a Oreo è la copertura multi-linguaggio: essendo stati pre-addestrati su sei linguaggi simultaneamente, possono essere fine-tuned su dataset eterogenei senza essere vincolati a Java. Tuttavia, studi empirici evidenziano che la capacità di generalizzazione di CodeBERT decresce significativamente quando il modello viene valutato su frammenti di codice con funzionalità diverse da quelle usate durante l'addestramento, con un calo marcato nell'F1 score [26]. Questo limite, unito alla necessità di risorse computazionali significative per il fine-tuning, rappresenta il principale punto di attenzione per l'utilizzo di questi modelli in contesti accademici reali.

4.1.7 CodeLlama¹¹ e StarCoder2¹²

CodeLlama e StarCoder2 sono Large Language Models open source specializzati per il codice, self-hostabili localmente, e rappresentano i candidati naturali per applicare l'approccio LLM-based descritto nella Sezione 3.7 in un contesto accademico che non dipenda da API commerciali esterne.

Come per CodeBERT e GraphCodeBERT, non si tratta di strumenti autonomi per la SCPD: i loro pesi sono rilasciati pubblicamente su HuggingFace e il loro utilizzo per la rilevazione del plagio richiede la costruzione di una pipeline sperimentale basata su prompt engineering, come discusso nella Sezione 3.7.1.

CodeLlama è una famiglia di LLM sviluppata da Meta AI nel 2023, costruita tramite fine-tuning di Llama 2 su codice sorgente. La famiglia è disponibile in taglie da 7B, 13B, 34B e 70B parametri, addestrate su sequenze

di 16.384 token con supporto a contesti fino a 100.000 token. Per il task della SCPD la variante rilevante è CodeLlama-Instruct, allineata tramite instruction tuning per seguire istruzioni in linguaggio naturale, rendendola adatta a ricevere prompt strutturati e restituire risposte argomentate. I modelli sono rilasciati sotto una licenza custom di Meta che ne permette l'uso commerciale e la ricerca, e sono eseguibili localmente su singola GPU a partire dalla variante 7B.

4.2 Strumenti Commerciali e Piattaforme Integrate

4.2.1 MOSS (Measure Of Software Similarity)¹³

Sviluppato nel 1997 all'Università di Stanford, MOSS è probabilmente lo strumento di rilevazione del plagio più diffuso a livello accademico.

MOSS utilizza un approccio basato sui token. Dopo aver effettuato una procedura di *lexing* sul codice e rimosso spazi e commenti, lo strumento non utilizza algoritmi di string matching completi (come GST), ma implementa una tecnica proprietaria basata sul **Winnowing** (Document Fingerprinting). Il codice viene diviso in *k-grammi* di token; di questi viene calcolato l'hash e solo un sottoinsieme selezionato (la fingerprint) viene memorizzato e confrontato.

Questo approccio rende MOSS estremamente scalabile e veloce, permettendo di confrontare migliaia di file in pochi secondi. È robusto contro l'offuscamento lessicale (renaming) e il riordinamento del codice, poiché le «impronte» sono invarianti rispetto alla posizione. Un numero molto vasto di linguaggi è supportato da MOSS, tra questi C, C++, Java e Python.

MOSS viene fornito come servizio online (tramite script di invio al server di Stanford) e non è open-source. Inoltre, come tutti i sistemi token-based, può essere ingannato da pesanti modifiche strutturali o dall'inserimento massiccio di codice «rumore» che altera i *k-grammi*. Inoltre, studi recenti hanno rilevato una grave criticità legata all'Intelligenza Artificiale generative: MOSS può essere facilmente eluso utilizzando LLM pre-addestrati. È stato dimostrato, ad esempio, che studenti che utilizzano modelli come GPT-J per completare assegnamenti di programmazione introduttiva riescono a bypassare completamente i controlli di MOSS [27]. Questo accade poiché il codice generato da tali modelli risulta sufficientemente diversificato nella struttura e manca di quei «segnali» tipici che i sistemi algoritmici tradizionali utilizzano per identificare il plagio manuale.

¹¹<https://ollama.com/library/codellama>

¹²<https://ollama.com/library/starcoder2>

¹³<https://theory.stanford.edu/~aiken/moss/>

4.2.2 CodeMatch

CodeMatch è uno strumento integrato nella suite commerciale CodeSuite, sviluppata originariamente da Bob Zeidman e attualmente gestita dalla Software Analysis & Forensic Engineering (S.A.F.E.) Corporation per la rilevazione di violazioni di copyright. A differenza di molte soluzioni accademiche open source, è un software proprietario distribuito come file eseguibile binario compatibile esclusivamente con sistemi Windows. L'utilizzo è gratuito per l'analisi di volumi di codice sorgente inferiori a 1 Mbyte, mentre richiede l'acquisto di una licenza per quantità superiori.

L'approccio algoritmico di CodeMatch si basa sull'analisi simultanea di quattro diversi livelli di similarità. Per identificare le copie, lo strumento misura: la correlazione delle singole istruzioni (statement correlation), la correlazione di commenti e stringhe di testo, la correlazione degli identificatori e la correlazione delle sequenze di istruzioni.

Lo strumento permette il confronto automatico di migliaia di file contenuti in cartelle e sottocartelle. Tra le sue caratteristiche principali include la generazione di report che classificano le coppie di file più correlate, insieme ai loro elementi specifici che hanno segnalato il plagio. Lo strumento è estremamente versatile e supporta ben 36 linguaggi di programmazione.

Test empirici condotti in letteratura su dataset di riferimento (come algoritmi in Java e in C modificati con varie tecniche di plagio) confermano una buona accuratezza generale dello strumento [3]. Tuttavia, mostra una vulnerabilità specifica contro l'offuscamento lessicale (il 3° tipo di plagio), ovvero la ridenominazione massiccia degli identificatori. Poiché i suoi algoritmi tentano di trovare corrispondenze dirette tra i nomi delle variabili, un'alterazione sistematica di questi elementi abbassa inevitabilmente le percentuali di rilevamento. Questo limite diventa particolarmente marcato quando si analizzano file di grandi dimensioni contenenti un elevato numero di identificatori

4.2.3 Codequiry

Codequiry si posiziona come una soluzione commerciale di fascia enterprise progettata per superare i limiti degli strumenti accademici tradizionali attraverso un approccio multi-livello alla rilevazione della similarità. A differenza di tool come MOSS, Codequiry adotta un motore di analisi proprietario denominato Zeus, basato sulla tecnologia Abstract Syntax Tree (AST), che permette di ignorare le modifiche puramente estetiche (offuscamento lessicale) come la ridenominazione di variabili, il cambiamento della formattazione o la manipolazione dei commenti per concentrarsi esclusivamente sulla struttura logica del programma [28].

Il sistema opera attraverso una «tripletta» di controlli simultanei: il Peer Check per il confronto interno tra le sottomissioni di un gruppo, il Database Check per il riscontro con repository storici e il Web Check. Quest'ultimo rappresenta uno dei punti di forza dello strumento, in quanto si avvale di un livello di indicizzazione su scala globale che interroga oltre 1 trilione (1T+) di sorgenti di codice e più di 100 milioni di repository pubblici, inclusi GitHub, GitLab, Stack Overflow e portali di tutoring come Chegg.

Dal punto di vista dell'efficacia, uno studio tecnico condotto dalla stessa piattaforma riporta metriche di performance elevate su un campione di 100 sottomissioni: un'accuratezza complessiva del 95%, con una precisione del 95,5% e un tasso di Recall (recupero) del 98,9%. Oltre ai report testuali, Codequiry introduce strumenti di visualizzazione avanzati, come i grafici di clustering 2D, che permettono ai docenti di identificare istantaneamente pattern di collaborazione illecita o reti di plagio all'interno di classi numerose [29].

4.2.4 LLM Commerciali: La famiglia GPT

L'introduzione di modelli linguistici di grandi dimensioni, in particolare GPT-4o (Generative Pre-trained Transformer) di OpenAI, rappresenta un cambio di paradigma rispetto ai sistemi di rilevazione deterministici discussi finora. A differenza degli strumenti basati su token o grafi, GPT non richiede una pipeline di analisi statica predefinita; la sua capacità di identificare il plagio emerge dalla vasta pre-esposizione a miliardi di parametri che includono codice sorgente e documentazione tecnica in molteplici linguaggi.

Il compito di Source Code Plagiarism Detection (SCPD) viene affrontato da questi modelli prevalentemente in modalità zero-shot, ovvero senza necessità di un addestramento specifico sul problema, ma attraverso l'utilizzo di prompt strutturati in linguaggio naturale che descrivono il task e i frammenti da confrontare.

L'efficacia di questo approccio è stata validata empiricamente [16]. I risultati mostrano che GPT-4o è in grado di raggiungere un'accuratezza complessiva del 78,70%, con un F1 score di 86,97%. Il valore aggiunto di questo modello risiede nella capacità di superare le barriere sintattiche che limitano strumenti come JPlag o MOSS: GPT-4o dimostra prestazioni superiori nel rilevamento dei cloni di Tipo 4 (variazioni semantiche profonde), riuscendo a identificare la similarità logica anche quando l'algoritmo viene completamente riscritto o tradotto tra linguaggi di programmazione differenti (plagio cross-language).

Tuttavia, l'adozione di GPT in ambito accademico e legale deve affrontare sfide strutturali. Sebbene il modello offra una spiegabilità nativa — producendo ragionamenti in linguaggio naturale che motivano il verdetto di plagio — esso rimane un sistema non deterministico. Lo stesso prompt

può generare risultati leggermente diversi se eseguito più volte, complicando la piena riproducibilità degli esperimenti richiesti in ambito forense. Inoltre, persiste il rischio di falsi positivi, poiché il modello tende a segnalare come sospetti alcuni pattern di implementazione estremamente comuni, come gli algoritmi di ordinamento standard, anche in assenza di un reale riutilizzo non autorizzato.

4.3 Tabella Comparativa degli Strumenti

A completamento della rassegna degli strumenti, vengono presentate due tabelle di sintesi. La prima si focalizza sulle soluzioni accademiche e open source, mentre la seconda analizza le piattaforme commerciali e i modelli linguistici di grandi dimensioni. Questo confronto permette di valutare rapidamente il compromesso tra potenza algoritmica, facilità d'uso e capacità di spiegazione del verdetto.

Tool	Linguaggi	Tipi di Clone	Spieg.	Esperienza Utente
JPlag	19	Tipo 1, 2, 3 (Parziale)	Alta	UI locale, grafici similarità e cluster
SIM	9	Tipo 1, 2	Alta	CLI, output testuale semplice
Plaggie	1 (Java)	Tipo 1, 2	Alta	CLI standalone, report di base
Dolos	Agnostico (Tree-sitter)	Tipo 1, 2, 3 (Parziale)	Alta	Web app, dashboard, grafici 2D
Oreo	Java	Tipo 1, 2, 3 e «Twilight Zone»	Nulla	CLI e modello Deep Learning, niente report oltre la percentuale di similarità
CodeBERT	6	Tutti	Nulla	Assente, richiede la creazione autonoma di una pipeline
CodeLlama	Multi-linguaggio (Universale)	Tutti (Tipo 1-4)	Altissima	Output testuale: Richiede pipeline per generare report

Tabella 2: Confronto tra strumenti accademici e open source

Tool	Linguaggi	Tipi di Clone	Spieg.	Esperienza Utente
MOSS	25	Tipo 1, 2, 3 (Parziale)	Alta	Servizio web, report HTML remoto
Code-Match	50+	Tipo 1, 2, 3	Altissima	App Windows, report multi-livello forensic-oriented
Code-quiry	65+	Tutti (1-4)	Alta	Altissima (SaaS). Dashboard interattive, funzionalità di esportazione
GPT-4o	Universale (Cross-linguaggio)	Tutti (Tipo 1-4)	Altissima	Chat/API, ragionamento discorsivo

Tabella 3: Confronto tra strumenti commerciali

5 Metodologie di Valutazione e Dataset

5.1 Dataset di Riferimento

5.2 Metriche di Performance

5.3 Robustezza

6 Nuove Sfide: L'Era dell'AI Generativa

6.1 Il «Plagio» da AI

6.2 Rilevamento AI-Generated Code

6.3 Differenza tra rilevazione di plagio (copia) e rilevazione di AI (generazione)

7 Conclusioni e Sviluppi Futuri

Bibliografia

[1] L. Prechelt, G. Malpohl, e M. Philippsen, «Finding Plagiarisms among a Set of Programs with JPlag», *Journal of Universal Computer Science*, 2002.

[2] G. Cosma, «An Approach to Source-Code Plagiarism Detection and Investigation Using Latent Semantic Analysis», Doctoral dissertation, 2008.

[3] V. T. Martins, D. Fonte, P. R. Henriques, e D. da Cruz, «Plagiarism Detection: A Tool Survey and Comparison», 2014.

[4] S. Yetthapu, «Source Code Plagiarism Detection Using JPlag & Stack Overflow Data», 2023.

[5] J. Hage, P. Rademaker, e N. van Vugt, «A comparison of plagiarism detection tools», technical report UU-CS-2010-015, 2010.

[6] B. Prado, K. Bispo, e R. Andrade, «X9: An Obfuscation Resilient Approach for Source Code Plagiarism Detection in Virtual Learning Environments», in *Proceedings of the 20th International Conference on Enterprise Information Systems (ICEIS 2018)*, 2019.

[7] S. Ko, J. Choi, e H. Kim, «COAT: Code Obfuscation Tool to evaluate the performance of code plagiarism detection tools», in *International Conference on Software Security and Assurance (ICSSA)*, 2017.

[8] P. G. Hamer e G. D. Frewin, «M. H. Halstead's software science: A critical examination», in *Proceedings of the 6th international conference on Software engineering*, 1982.

[9] T. J. McCabe, «A complexity measure», *IEEE Transactions on software Engineering*, 1976.

- [10] J. W. Hunt e T. G. Szymanski, «A Fast Algorithm for Computing Longest Common Subsequences», *Communications of the ACM*, vol. 20, fasc. 5, pp. 350–353, mag. 1977.
- [11] M. J. Wise, «String Similarity via Greedy String Tiling and Running Karp-Rabin Matching», Technical Report, 1993.
- [12] W. Wang, G. Li, B. Ma, X. Xia, e Z. Jin, «Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree», 2020. [Online]. Disponibile su: <https://arxiv.org/abs/2002.08653>
- [13] F. P. Viertel, W. Brunotte, D. Strüber, e K. Schneider, «Detecting Security Vulnerabilities using Clone Detection and Community Knowledge», in *Proceedings of the 14th International Conference on Risks and Security of Internet and Systems (CRiSIS)*, Springer, 2019.
- [14] C. Liu, C. Chen, J. Han, e P. S. Yu, «GPLAG: Detection of Software Plagiarism by Program Dependence Graph Analysis», in *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '06)*, New York, NY, USA: Association for Computing Machinery, 2006.
- [15] Z. Feng, D. Guo, D. Tang, e others, «CodeBERT: A Pre-Trained Model for Programming and Natural Languages», in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.
- [16] W. Brach, K. Košťál, e M. Ries, «Can Large Language Model Detect Plagiarism in Source Code?», in *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, 2024, pp. 370–377. doi: 10.1109/FLLM63129.2024.10852497.
- [17] T. Sağlam, S. Hahner, L. Schmid, e E. Burger, «Obfuscation-Resilient Software Plagiarism Detection with JPlag», 2024.
- [18] D. Grune e M. H. Huntjens, «SIM: A text similarity tester», technical report, 1989. [Online]. Disponibile su: https://dickgrune.com/Programs/similarity_tester/
- [19] T. P. Gondaliya, H. D. Joshi, e H. Joshi, «Source Code Plagiarism Detection 'SCPDet': A Review».
- [20] A. Ahtiainen, S. Surakka, e M. Rahikainen, «Plagie: GNU-licensed source code plagiarism detection engine for Java exercises», 2006.
- [21] R. Maertens, P. Dawyndt, e B. Mesuere, «Dolos 2.0: Towards Seamless Source Code Plagiarism Detection in Online Learning Environments», 2023.
- [22] R. Maertens *et al.*, «Dolos: Language-agnostic plagiarism detection in source code», 2022.
- [23] R. Maertens *et al.*, «Discovering and exploring cases of educational source code plagiarism with Dolos», 2024.
- [24] V. Saini, F. Farmahinifarahani, Y. Lu, P. Baldi, e C. Lopes, «Oreo: Detection of clones in the twilight zone», 2018.
- [25] D. Guo *et al.*, «GraphCodeBERT: Pre-training Code Representations with Data Flow», 2021.
- [26] T. Sonnekalb, B. Gruner, C.-A. Brust, e P. Mäder, «Generalizability of Code Clone Detection on CodeBERT», 2022.
- [27] S. Biderman e E. Raff, «Fooling MOSS Detection with Pretrained Language Models», 2022.
- [28] Codequiry, «Code Plagiarism Checker and AI Detector». 2026.
- [29] Codequiry Team, «Evaluation of Codequiry's Effectiveness in Detecting Code Plagiarism», Technical Report, 2023.